Cryptographie R309

Code correcteur d'erreur de Hamming

Codage, décodage et correction d'une erreur — TP6

Coder un message en un mot de code de 2^n bits, calculer les bits de contrôle par XOR, décoder en localisant l'erreur grâce au syndrome, puis étendre le code par un bit de parité pour détecter les erreurs doubles : les treize questions du sujet sont traitées, expliquées et vérifiées par le script de test fourni.



Un mot de code de l'énoncé (n = 4) : les positions de contrôle 0, 1, 2, 4 et 8 sont en bleu, les onze bits du message occupent les autres positions.

Auteur	Johan Polsinelli	**Classe**	BUT Réseaux & Télécommunications
Date	9 novembre 2026	**Fichiers**	hamming.py · test-hamming.py · verification.py

13 / 13 fonctions validées par test-hamming.py

Sommaire

1. Principe du code de Hamming 3
1.1 Le syndrome, ou « pourquoi le XOR suffit » 3
2. Codage : questions 1 à 8 4
Question 1 — int2bits : écrire un entier sur n bits 4
Question 2 — bits2int et réciprocité 5
Question 3 — generate_message : tirage aléatoire 6
Question 4 — formatage : insérer les positions de contrôle 6
Question 5 — XOR de deux tableaux 8
Question 6 — test_hamming : calcul du syndrome 8
Question 7 — set_correctors : fixer les bits de contrôle 9
Question 8 — codage_hamming : la chaîne complète 10
Question 9 — vérification : le syndrome d'un mot de code est nul 10
3. Décodage et correction : questions 10 à 13 11
Question 10 — swap et decodage : localiser puis corriger 11
Question 11 — parite 13
Question 12 — codage_hamming2 : ajout du bit de parité 14
Question 13 — decodage2 : corriger une erreur, détecter deux erreurs 15
4. Validation du programme 16
4.1 Script de test 16
4.2 Conformité au cahier des charges 17
4.3 Complexités 17
5. Bilan 17
Annexe A — code complet de hamming.py 18
Annexe B — traces d'exécution 22
Annexe C — reproductibilité 24

1. Principe du code de Hamming

Un canal de transmission n'est jamais parfait : un bit peut être inversé pendant le transport (bruit, interférence, lecture d'un support). Un **code correcteur d'erreur** ajoute de la redondance au message pour que le récepteur puisse non seulement **détecter** une altération, mais aussi la **corriger** sans avoir à demander une retransmission.

Le code de Hamming étudié ici est un code **linéaire** très simple, dont toute la mécanique repose sur une seule opération : le **ou exclusif** (XOR). Un mot de code est un tableau de 2^n bits numérotés de 0 à $2^n - 1$:

- la position 0 et les positions 1, 2, 4, 8, …, 2^{n-1} (les puissances de 2) sont des **positions de contrôle** ; elles ne transportent pas d'information, la position 0 étant réservée au bit de parité de la question 12 ;
- les $2^n - (n + 1)$ autres positions contiennent les bits du message, **dans l'ordre**, du plus petit indice libre au plus grand ;
- les n bits de contrôle sont calculés pour que le XOR des positions de tous les 1 du mot de code soit nul.

Grandeur	Notation	Valeur
Nombre de bits de contrôle	n	3, 4, 5, …
Taille du message	$L = 2^n - (n + 1)$	4, 11, 26, 57, 120, …
Taille du mot de code	2^n	8, 16, 32, 64, 128, …
Positions de contrôle	0, 1, 2, 4, …, 2^{n-1}	n + 1 positions
Positions du message	les autres	L positions

n	Bits du message	Bits du code	Rendement $L / 2^n$	Bits de contrôle
3	4	8	0,500	0, 1, 2, 4
4	11	16	0,688	0, 1, 2, 4, 8
5	26	32	0,813	0, 1, 2, 4, 8, 16
6	57	64	0,891	0, 1, …, 32
7	120	128	0,938	0, 1, …, 64
8	247	256	0,965	0, 1, …, 128

Le rendement augmente avec n : la redondance devient négligeable pour les grands blocs, mais un bloc plus long est aussi plus exposé aux erreurs multiples. C'est ce compromis qui fixe la taille des blocs dans les systèmes réels.

1.1 Le syndrome, ou « pourquoi le XOR suffit »

On appelle **syndrome** du mot c le XOR de toutes les positions i où $c_i = 1$:

> $$s(c) = \oplus \{ \text{ i tel que } c_i = 1 \}$$

Comme chaque position i s'écrit de façon unique en binaire sur n bits, $i = \Sigma_k\ i_k \cdot 2^k$, le XOR se lit coordonnée par coordonnée : la k-ième coordonnée du syndrome est la somme modulo 2 des k-ièmes bits des positions occupées.

Trois propriétés, qui structurent tout le TP :

1. **Un mot de code valide a un syndrome nul** : c'est exactement la condition de définition des bits de contrôle.
2. **Si un seul bit est inversé en position p, le syndrome vaut p** : l'ancien syndrome (nul) est XORé avec p, et $p \neq 0$. Le syndrome localise donc l'erreur, ce qui permet de la corriger par une seconde inversion.
3. **Si deux bits sont inversés en positions p et q, le syndrome vaut p ⊕ q** : il est non nul, mais il désigne une position qui n'est pas celle de l'erreur. Le décodeur « corrige » alors un bit sain et produit un message faux, **sans le signaler**. C'est la limite du code de Hamming simple, et la raison d'être des questions 11 à 13.

Le XOR est ainsi à la fois le **calcul des bits de contrôle** (au codage) et le **calcul du syndrome** (au décodage) : les deux moitiés du TP réutilisent la même primitive.

2. Codage : questions 1 à 8

ÉNONCÉ

On considère un message m de $2^n - (n + 1)$ bits. Le codage consiste à insérer dans le message les n bits de contrôle et la position 0 réservée, puis à calculer les bits de contrôle.

Question 1 — int2bits : écrire un entier sur n bits

ÉNONCÉ

Écrire une fonction `int2bits` qui prend en entrée un entier a et un nombre de bits n et renvoie un tableau de n bits représentant a, le premier élément étant le bit de poids faible. Exemple : `int2bits(13, 6)` doit renvoyer `[1, 0, 1, 1, 0, 0]`.

RÉPONSE

L'écriture demandée est celle de la décomposition en base 2, mais rangée dans l'ordre **croissant** des poids :

$$a = b_0 \cdot 2^0 + b_1 \cdot 2^1 + \ldots + b_{n-1} \cdot 2^{n-1} \text{ avec } b_k = \lfloor a / 2^k \rfloor \bmod 2.$$

Le bit de poids k s'obtient donc par un décalage à droite de k rangs suivi d'un masque sur le bit de poids faible : c'est exactement `(a >> k) & 1`. La fonction se résume à une liste en compréhension, sans boucle explicite ni variable temporaire.

Attention à la convention : `int2bits(13, 6)` n'est **pas** l'écriture binaire usuelle de 13 (1101) lue de gauche à droite, mais son miroir complété à 6 bits. Seul l'ordre des cases est inversé, pas la valeur des bits.

```
def int2bits(a, n):
    """Representation de a sur n bits, poids faible en premier.

    Entrees : a entier (0 <= a < 2^n), n nombre de bits
    Sortie  : liste de n bits, le k-ieme element etant le bit de poids 2^k

    Exemple : int2bits(13, 6) vaut [1, 0, 1, 1, 0, 0].
    """
    return [(a >> k) & 1 for k in range(n)]
```

Appel	Résultat	Décomposition
`int2bits(13, 6)`	`[1, 0, 1, 1, 0, 0]`	1 + 0 + 4 + 8 + 0 + 0 = 13
`int2bits(0, 4)`	`[0, 0, 0, 0]`	0
`int2bits(15, 4)`	`[1, 1, 1, 1]`	1 + 2 + 4 + 8 = 15
`int2bits(6, 3)`	`[0, 1, 1]`	2 + 4 = 6
`int2bits(45, 6)`	`[1, 0, 1, 1, 0, 1]`	1 + 4 + 8 + 32 = 45

Coût : **O(n)** opérations élémentaires, un décalage et un masque par bit. La fonction suppose seulement $0 \leq a < 2^n$; au-delà, les bits de poids fort sont silencieusement tronqués, ce qui est le comportement attendu d'une écriture « sur n bits ».

Question 2 — bits2int et réciprocité

ÉNONCÉ

Écrire une fonction `bits2int` qui prend en entrée un tableau de bits et renvoie l'entier qu'il représente. Vérifier que les deux fonctions sont bien réciproques l'une de l'autre.

RÉPONSE

C'est la somme pondérée des bits, le bit d'indice k étant affecté du poids 2^k :

$$\text{bits2int}(b) = \Sigma_k\, b_k \cdot 2^k, \text{ la somme portant sur } k = 0, 1, \ldots, n-1.$$

Le code utilise une somme en compréhension et le décalage `b << k`, équivalent à $b \cdot 2^k$ mais sans multiplication explicite.

Les deux fonctions sont réciproques au sens suivant : pour tout entier a de $\{0, \ldots, 2^n - 1\}$ et tout mot b de n bits,

$$\text{bits2int}(\text{int2bits}(a, n)) = a \text{ et } \text{int2bits}(\text{bits2int}(b), n) = b.$$

Cela découle de l'**unicité de l'écriture en base 2** : la décomposition $a = \Sigma\, b_k\, 2^k$ avec $b_k \in \{0, 1\}$ est unique, donc l'application $b \mapsto \Sigma\, b_k\, 2^k$ est une bijection de $\{0, 1\}^n$ sur $\{0, \ldots, 2^n - 1\}$, et la fonction int2bits en est la bijection réciproque. Autrement dit, le mot de n bits est la décomposition, et l'entier en est l'évaluation.

```
def bits2int(bits):
    """Entier represente par un tableau de bits (poids faible en premier).

    Entrees : bits liste de 0 et de 1
    Sortie  : somme des bits[k] * 2^k ; c'est la reciproque de int2bits
```

```
    """
    return sum(b << k for k, b in enumerate(bits))
```

La vérification exhaustive confirme la bijection sur 8 bits : les 256 entiers et les 256 mots de 8 bits sont traités sans un seul écart.

```
bits2int([1, 0, 1, 1, 0, 0]) = 13
bits2int(int2bits(a, 8)) == a                 pour a = 0..255 : True
int2bits(bits2int(x)) == x   pour les 256 mots de 8 bits        : True
```

Coût : **O(n)**. Remarque : la réciprocité n'est garantie que si l'entier tient sur n bits ; `bits2int` accepte, lui, n'importe quelle longueur de tableau, ce qui est utile pour les messages de 11, 26 ou 57 bits.

Question 3 — generate_message : tirage aléatoire

ÉNONCÉ

Écrire une fonction `generate_message` qui prend en entrée un nombre n et renvoie un tableau aléatoire de bits de taille $2^n - (n + 1)$.

RÉPONSE

La taille est celle des positions libres (ni 0, ni puissance de 2). Chaque bit est tiré indépendamment avec `randint(0, 1)`, ce qui donne une loi **uniforme** sur les 2^L messages possibles : tous les messages ont la même probabilité, condition nécessaire pour que les tests statistiques du TP aient un sens.

La fonction ne contient aucun cas particulier : pour n = 3 elle renvoie 4 bits, pour n = 8 elle en renvoie 247.

```
def generate_message(n):
    """Message aleatoire de taille 2^n - (n + 1).

    Entrees : n entier
    Sortie  : liste de 2^n - (n + 1) bits tires au hasard
    """
    return [randint(0, 1) for _ in range(2 ** n - (n + 1))]
```

```
n = 3 -> 4 bits  : [0, 0, 1, 1]
n = 4 -> 11 bits : [1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0]
n = 5 -> 26 bits : [0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 1,
        1]
200 tirages pour n = 5 : 200 messages distincts
```

Question 4 — formatage : insérer les positions de contrôle

ÉNONCÉ

Écrire une fonction `formatage` qui prend en entrée un message m, sa taille définie par n, et renvoie un tableau de bits de taille 2^n en insérant un 0 à toutes les positions qui sont une puissance de 2 ainsi qu'à la position 0. Le tableau m ne doit pas être modifié.

RÉPONSE

La question cache deux difficultés.

a) Reconnaître une position de contrôle. Les positions de contrôle sont 0 et les puissances de 2. Pour i > 0, i est une puissance de 2 si et seulement si son écriture binaire ne contient qu'un seul 1, c'est-à-dire si `i & (i - 1) == 0` : retirer le 1 de poids faible éteint alors tous les bits. Ce test est en temps constant, sans logarithme ni division. La position 0 est traitée à part, car `0 & -1` vaut 0 alors que 0 n'est pas une puissance de 2.

b) Ne pas modifier l'argument. Le texte insiste : `formatage` construit une **nouvelle** liste de 2^n zéros, puis y recopie le message aux bonnes positions. Aucune écriture n'est faite dans m, qui reste intact pour l'appelant.

L'insertion se fait en parcourant les positions libres dans l'ordre croissant et en les appariant avec les bits du message, eux aussi pris dans l'ordre : le k-ième bit du message se retrouve à la k-ième position libre. C'est exactement la convention du sujet.

```
def est_controle(i):
    """Vrai si la position i est reservee a un bit de controle."""
    return i == 0 or (i & (i - 1)) == 0


def positions_message(n):
    """Liste des positions de message pour un mot de code de 2^n bits."""
    return [i for i in range(2 ** n) if not est_controle(i)]


def formatage(m, n):
    """Insertion des bits de controle (mis a 0) dans le message."""
    formate = [0] * (2 ** n)
    for position, bit in zip(positions_message(n), m):
        formate[position] = bit
    return formate
```

Application au message de l'énoncé (n = 4) : les positions libres sont 3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, et l'on y recopie un à un les 11 bits du message.

Position	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Rôle	rés.	c	c	m	c	m	m	m	c	m	m	m	m	m	m	m
Valeur	0	0	0	1	0	0	0	1	0	0	1	1	1	0	1	1

```
message   : [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]
formatage : [0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1]
attendu   : [0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1]
identique : True
```

Coût : **$O(2^n)$**, une écriture par position, plus la construction de la liste des positions libres.

Question 5 — XOR de deux tableaux

ÉNONCÉ

Implémenter une fonction `XOR` qui prend en entrée deux tableaux de même taille et renvoie le tableau obtenu en réalisant le ou exclusif sur chaque paire d'éléments correspondants.

RÉPONSE

Le ou exclusif bit à bit est déjà l'opérateur `^` de Python sur les entiers 0 et 1 ; il suffit de l'appliquer composante par composante. On utilise `zip`, qui s'arrête à la plus courte des deux listes : la fonction est donc sûre même si les tailles diffèrent, mais l'usage prévu est bien celui de tableaux de même longueur.

La table de vérité de `^` sur {0, 1} est celle d'une **addition modulo 2** : $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$. C'est cette structure de groupe $(\mathbb{Z}/2\mathbb{Z})^n$ qui fait fonctionner tout le code : le syndrome est un XOR de positions, donc une somme dans ce groupe.

```
def XOR(m, p):
    """Ou exclusif terme a terme de deux tableaux de meme taille."""
    return [a ^ b for a, b in zip(m, p)]
```

a	b	a ⊕ b
`[1, 0, 1, 1]`	`[0, 1, 1, 0]`	`[1, 1, 0, 1]`
`[0, 0, 0]`	`[0, 0, 0]`	`[0, 0, 0]`
`[1, 1]`	`[1, 1]`	`[0, 0]`

Coût : **$O(2^n)$**.

Question 6 — test_hamming : calcul du syndrome

ÉNONCÉ

Écrire une fonction `test_hamming` qui prend en entrée un tableau m de 2^n bits ainsi que n et renvoie le tableau de n bits obtenu en appliquant XOR à l'ensemble des tableaux `int2bits(i, n)` pour tous les indices i tels que m[i] = 1.

RÉPONSE

Le syndrome est la somme (dans le groupe des n-uplets de bits) des écritures binaires des positions occupées. On part du vecteur nul `[0] * n` et l'on XOR successivement `int2bits(i, n)` pour chaque 1 rencontré : l'ordre du parcours n'a aucune importance, le XOR étant associatif et commutatif.

C'est ici que la question 1 prend tout son sens : `int2bits(i, n)` fournit les n coordonnées du syndrome, une par bit de contrôle. Le résultat est donc bien un vecteur de n bits, qui se lit comme l'entier `bits2int(syndrome)` — la position de l'erreur, cf. question 10.

Vérification sur le message formaté de la question 4 : les 1 sont aux positions 3, 7, 10, 11, 12, 14, 15, et le XOR cumulé vaut 8.

```
def test_hamming(m, n):
    """Syndrome de Hamming du mot m, ecrit sur n bits."""
    syndrome = [0] * n
    for position, bit in enumerate(m):
        if bit == 1:
            syndrome = XOR(syndrome, int2bits(position, n))
    return syndrome
```

Étape	Position ajoutée	Valeur du XOR cumulé	En binaire (8 4 2 1)
1	3	3	0011
2	7	3 ⊕ 7 = 4	0100
3	10	4 ⊕ 10 = 14	1110
4	11	14 ⊕ 11 = 5	0101
5	12	5 ⊕ 12 = 9	1001
6	14	9 ⊕ 14 = 7	0111
7	15	7 ⊕ 15 = **8**	1000

```
positions des 1 de m  : [3, 7, 10, 11, 12, 14, 15]
XOR de ces positions  : 3 ^ 7 ^ 10 ^ 11 ^ 12 ^ 14 ^ 15 = 8
int2bits(8, 4)        : [0, 0, 0, 1]
test_hamming(m, 4)    : [0, 0, 0, 1]       (soit 8)
test_hamming(code, 4) : [0, 0, 0, 0]       (mot de code valide)
```

Coût : **$O(n \cdot 2^n)$**, un XOR de n bits par position occupée. C'est la brique la plus coûteuse du TP, appelée une fois au codage et une fois au décodage.

Question 7 — set_correctors : fixer les bits de contrôle

ÉNONCÉ

Écrire une fonction `set_correctors` qui prend en entrée un tableau m de 2^n bits dont les bits correcteurs valent 0 ainsi que n, et renvoie un nouveau message où les bits correcteurs ont été définis. Le k-ième élément de `test_hamming(m, n)` est la valeur à donner au bit correcteur situé en position 2^k.

RÉPONSE

Le résultat est immédiat une fois remarqué que chaque bit de contrôle est **seul** sur sa coordonnée : la position 2^k a pour écriture binaire le vecteur e_k (un 1 en k-ième position, des 0 ailleurs). Elle n'influe donc que sur la k-ième coordonnée du syndrome.

Notons s = `test_hamming(m, n)` le syndrome du message formaté, dont les bits de contrôle valent encore 0. Poser le bit de contrôle d'indice 2^k à la valeur s_k, c'est ajouter au syndrome le vecteur e_k chaque fois que $s_k = 1$: le syndrome devient donc $s \oplus s = 0$. Les bits de contrôle sont exactement les coordonnées manquantes pour annuler le syndrome, ce qui est la version constructive de la propriété « un mot de code a un syndrome nul ».

La fonction travaille sur une **copie** de m (`list(m)`), pour ne pas altérer l'argument, comme le faisait `formatage`.

```
def set_correctors(m, n):
    """Calcul des bits de controle du mot formate m."""
    corrige = list(m)
    syndrome = test_hamming(corrige, n)
    for k in range(n):
        corrige[2 ** k] = syndrome[k]
    return corrige
```

```
formatage(m, 4)      : [0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1]
test_hamming(m, 4)   : [0, 0, 0, 1]        -> les bits 1, 2, 4 restent à 0, le bit 8 passe à 1
set_correctors(m, 4) : [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]
```

Coût : **O(n · 2ⁿ)**, dominé par l'appel à `test_hamming`.

Question 8 — codage_hamming : la chaîne complète

ÉNONCÉ

Écrire une fonction `codage_hamming` qui prend en entrée un message de $2^n - (n + 1)$ bits ainsi que la taille n et renvoie le message formaté avec les bits correcteurs définis.

RÉPONSE

Le codage est la composition des deux étapes précédentes : **formater** (insérer les positions de contrôle à 0), puis **corriger** (calculer ces bits). Écrire la fonction comme une composition plutôt que comme un long bloc de code rend la chaîne lisible et évite toute duplication : chaque étape reste testable séparément, ce qui est exactement ce que fait `test-hamming.py`.

On remarque que `codage_hamming` renvoie une nouvelle liste de 2^n bits : le message d'entrée n'est jamais modifié, propriété nécessaire pour que l'appelant puisse coder plusieurs fois le même message.

```
def codage_hamming(m, n):
    """Codage de Hamming d'un message de 2^n - (n + 1) bits."""
    return set_correctors(formatage(m, n), n)
```

```
message        : [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]
codage_hamming : [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]
attendu        : [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]
identique      : True
```

Coût : **O(n · 2ⁿ)**.

Question 9 — vérification : le syndrome d'un mot de code est nul

ÉNONCÉ

En théorie, après avoir mis à jour les bits correcteurs d'erreur, le test de Hamming `test_hamming` devrait donner un tableau de n zéros. Vérifier que c'est bien le cas sur un ou plusieurs messages traités avec le code Hamming.

RÉPONSE

La vérification a été menée de deux façons : de manière **exhaustive** sur tous les messages possibles pour n = 3 (16 messages) et n = 4 (2048 messages), puis de manière **aléatoire** à 1000 tirages par taille pour n allant de 3 à 7. Dans tous les cas, le syndrome du mot codé est le vecteur nul.

La raison est structurelle, et non accidentelle. Notons M l'ensemble des positions de message occupées par un 1 et C l'ensemble des positions de contrôle fixées à 1. Par construction (question 7), la k-ième coordonnée du syndrome vaut

$$s_k = (\text{nombre de } i \in M \text{ dont le } k\text{-ième bit vaut } 1) \bmod 2,$$

et l'on a posé $c_{2^k} = s_k$, c'est-à-dire que $2^k \in C$ exactement quand $s_k = 1$. La k-ième coordonnée du syndrome du mot complet est donc $s_k \oplus s_k = 0$ pour chaque k. Le syndrome est nul coordonnée par coordonnée, donc le mot obtenu est bien un mot de code.

On en déduit aussi le **nombre de mots de code** : l'application message ↦ mot de code est linéaire et injective (les positions de message sont recopiées telles quelles, donc le message se relit dans le code), d'où exactement $2^{2^n - (n+1)}$ mots de code, soit 1 mot de code sur 2^{n+1} mots binaires possibles. Pour n = 4 : 2048 mots de code sur 65 536 mots de 16 bits.

n	Taille du message	Taille du code	Tests aléatoires	Syndromes nuls	Mots de code
3	4	8	1000	1000	16
4	11	16	1000	1000	2048
5	26	32	1000	1000	33 554 432
6	57	64	1000	1000	2^{57}
7	120	128	1000	1000	2^{120}

```
 n  taille m  taille code  tests  syndromes nuls
 3         4            8   1000            1000
 4        11           16   1000            1000
 5        26           32   1000            1000
 6        57           64   1000            1000
 7       120          128   1000            1000

Mots de code distincts obtenus par énumération exhaustive : n=3 : 16, n=4 : 2048
```

La vérification confirme donc le résultat théorique : **tout message codé par** `codage_hamming` **est un mot de code valide**, et le code ne « gaspille » pas plus de redondance que prévu.

3. Décodage et correction : questions 10 à 13

Question 10 — swap et decodage : localiser puis corriger

ÉNONCÉ

Écrire une fonction `swap` qui prend en entrée un message m et un indice k et renvoie une copie de m dont le k-ième bit a été inversé (m lui-même n'est pas modifié). Vérifier que si on change un bit d'un message codé, le test de Hamming renvoie un résultat non nul et donne la position de l'erreur. Écrire une fonction `decodage` qui prend en entrée un message codé m de taille 2^n ainsi que n et renvoie le message décodé de taille $2^n - (n + 1)$, corrigé si une erreur est détectée.

RÉPONSE

swap est volontairement une fonction pure : elle copie la liste avant d'inverser le bit k (`1 - bit`), pour que l'appelant puisse fabriquer un message fautif sans perdre l'original — c'est ce dont les tests ont besoin pour simuler une erreur de transmission.

Localisation. Si le mot transmis est c (syndrome nul) et que le bit p est inversé, le mot reçu c' ne diffère de c qu'en p. Le syndrome étant linéaire, $s(c') = s(c) \oplus p = p$. Le syndrome **est** la position de l'erreur, écrite en binaire sur n bits. C'est la propriété annoncée dans les rappels du sujet, et elle se vérifie expérimentalement sur les 16 positions possibles du code de l'énoncé.

decodage applique alors la règle : si le syndrome est nul, rien à faire ; sinon, on inverse le bit désigné (via `swap`) et l'on relit les positions de message dans l'ordre. Comme le syndrome tient sur n bits, sa valeur est toujours un indice valide entre 1 et $2^n - 1$: il n'y a pas de cas d'erreur à traiter.

```
def swap(m, k):
    """Copie de m dont le bit d'indice k a ete inverse."""
    copie = list(m)
    copie[k] = 1 - copie[k]
    return copie


def decodage(m, n):
    """Decodage d'un mot de code de 2^n bits, avec correction d'une erreur."""
    corrige = list(m)
    erreur = bits2int(test_hamming(corrige, n))
    if erreur != 0:
        corrige = swap(corrige, erreur)
    return [corrige[position] for position in positions_message(n)]
```

Bit inversé	Syndrome (entier)	Message décodé	Correct
aucun	0	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
1	1	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
2	2	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
3	3	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
4	4	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
5	5	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
6	6	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui

Bit inversé	Syndrome (entier)	Message décodé	Correct
7	7	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
8	8	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
9	9	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
10	10	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
11	11	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
12	12	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
13	13	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
14	14	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui
15	15	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	oui

Sur les 16 erreurs simples simulées, les 16 sont corrigées : le syndrome redonne exactement l'indice fautif, et le message décodé est identique à l'original. Une erreur sur une position de contrôle (0, 1, 2, 4, 8) est corrigée de la même façon, puisqu'elle est traitée comme n'importe quelle autre position.

Ce que devient une erreur double

Le code de Hamming simple corrige **une** erreur, mais ne sait pas en détecter deux : $s(c') = p \oplus q \neq 0$, et le décodeur inverse alors un bit sain. Le message produit est faux, sans aucun signal d'alarme.

Bits inversés	Syndrome	Message décodé	Résultat
1 et 2	3	[0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	message faux
0 et 1	1	[1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]	message correct (par chance)
3 et 5	6	[0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1]	message faux
8 et 9	1	[1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1]	message faux
14 et 15	1	[1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0]	message faux

La ligne « 0 et 1 » est instructive : lorsque l'une des deux erreurs tombe sur la position 0, qui ne participe pas au syndrome, la correction « fonctionne » fortuitement — le bit 0 est en effet hors du message. C'est précisément la faille que corrige le bit de parité des questions suivantes.

Question 11 — parite

> **ÉNONCÉ**
> Écrire une fonction `parite` qui prend en entrée un message et renvoie sa parité, c'est-à-dire 0 si le message contient un nombre pair de 1, et 1 sinon.

> **RÉPONSE**
> La parité est la somme des bits modulo 2, c'est-à-dire le XOR de tous les bits : `sum(m) % 2`. C'est encore une opération dans $\mathbb{Z}/2\mathbb{Z}$, cette fois sur la **totalité** du mot et non sur une coordonnée.
>
> L'intérêt de ce bit est qu'il réagit différemment selon le nombre d'erreurs : inverser un bit change la parité (de pair à impair), en inverser deux la laisse inchangée. Combiné au syndrome, il permet donc de distinguer une erreur d'un nombre pair d'erreurs.

```
def parite(m):
    """Parite d'un tableau de bits : 0 si le nombre de 1 est pair, 1 sinon."""
    return sum(m) % 2
```

Message	Nombre de 1	Parité
`[0, 0, 0]`	0	0
`[1, 0, 1, 1]`	3	1
`[1, 1, 0, 1, 1]`	4	0

Question 12 — codage_hamming2 : ajout du bit de parité

ÉNONCÉ

Écrire une fonction `codage_hamming2` similaire à `codage_hamming` mais qui utilise le bit 0 pour réaliser un test de parité : il est choisi de sorte que le message codé complet contienne un nombre pair de 1.

RÉPONSE

La position 0, restée libre jusqu'ici, devient un **bit de parité globale**. On commence par coder normalement (les bits de contrôle sont calculés sans tenir compte de la position 0, qui vaut encore 0), puis on pose c_0 = parité du reste : le nombre total de 1 devient alors pair, puisque ajouter $c_0 = 1$ lorsque le nombre de 1 est impair le rend pair, et que l'ajout de 0 ne change rien.

Une autre rédaction, équivalente, consiste à dire que le code obtenu est le **code de Hamming étendu** : chaque mot de code a un syndrome nul et une parité nulle. Il y a donc un bit de redondance de plus, et la distance minimale du code passe de 3 à 4, ce qui autorise la détection des erreurs doubles.

```
def codage_hamming2(m, n):
    """Codage de Hamming avec un bit de parite globale en position 0."""
    code = codage_hamming(m, n)
    code[0] = parite(code)
    return code
```

Premier exemple, où le mot de code a déjà un nombre pair de 1 : le bit de parité reste à 0 et le mot est identique à celui de la question 8.

```
message              : [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]
codage_hamming(m, 4) : [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]   (8 bits à 1)
codage_hamming2(m, 4): [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]   (8 bits à 1)
syndrome             : [0, 0, 0, 0]
```

Second exemple, où le bit de parité est indispensable : le message réduit à `[1, 0, 0, …]` donne un mot de code à 3 bits à 1, donc impair.

```
message              : [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
codage_hamming(m, 4) : [0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]   (3 bits à 1, impair)
codage_hamming2(m, 4): [1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]   (4 bits à 1, pair)
syndrome             : [0, 0, 0, 0]
```

```
decodage2(c, 4)      : [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Question 13 — decodage2 : corriger une erreur, détecter deux erreurs

ÉNONCÉ

Écrire une fonction `decodage2` similaire à `decodage`, pour un message codé par `codage_hamming2`. Si `test_hamming` est nul, il n'y a pas d'erreur sur les bits 1 à $2^n - 1$: on renvoie le message décodé (une parité fausse signifie seulement que le bit 0 a été modifié). Si `test_hamming` n'est pas nul et que la parité est fausse, il y a une erreur : on la corrige et on renvoie le message décodé. Si `test_hamming` n'est pas nul mais que la parité est juste, il y a au moins deux erreurs : on ne peut pas corriger, et on renvoie `None`.

RÉPONSE

Le décodeur croise deux indicateurs : le **syndrome** (qui vaut 0 ou la position de l'erreur) et la **parité** (qui vaut 0 ou 1). Quatre combinaisons sont possibles, dont trois seulement apparaissent dans l'énoncé, la quatrième étant impossible :

Syndrome	Parité	Interprétation	Action
nul	juste	aucune erreur	renvoyer le message
nul	fausse	seule la position 0 a été modifiée (elle est hors syndrome)	renvoyer le message, qui est intact
non nul	fausse	une erreur, en position = syndrome	inverser ce bit, renvoyer le message
non nul	juste	au moins deux erreurs	renvoyer `None`
—	—	syndrome nul et parité juste **et** erreur ailleurs : impossible par construction	—

La justification tient en une phrase : **inverser un bit change la parité, en inverser deux ne la change pas**. Une erreur unique non nulle rend donc la parité fausse, tandis qu'une erreur double (ou toute erreur d'arité paire) laisse la parité juste. Le couple (syndrome, parité) sépare ainsi proprement les deux situations, et le décodeur peut refuser de corriger plutôt que de produire un message faux — c'est le comportement attendu d'un code **SECDED** (Single Error Correction, Double Error Detection).

Le cas « syndrome nul + parité juste » est le cas nominal. Le cas « syndrome nul + parité fausse » correspond à une erreur sur la seule position 0 : le message et les bits de contrôle sont intacts, il suffit d'ignorer le bit 0 — ce que fait naturellement la relecture des positions de message.

```
def decodage2(m, n):
    """Decodage du code de Hamming etendu, avec detection de deux erreurs."""
    mot = list(m)
    erreur = bits2int(test_hamming(mot, n))
    if erreur == 0:
        return [mot[position] for position in positions_message(n)]
    if parite(mot) != 0:
        return decodage(mot, n)
```

```
    return None
```

```
mot de code : [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]
0 erreur                -> [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]
1 erreur (16 positions) -> message retrouvé : True
2 erreurs (120 paires)  -> None : 120, autre : 0
```

Les trois comportements de l'énoncé sont donc vérifiés exhaustivement pour n = 4 : les 16 erreurs simples sont corrigées, et les 120 paires d'erreurs possibles (soit 16 × 15 / 2) sont toutes **détectées** sans exception. Le compromis est clair : on gagne la détection des erreurs doubles au prix d'un bit de redondance supplémentaire et de l'impossibilité de corriger ces erreurs doubles — un arbitrage classique en transmission, où l'on préfère demander une retransmission plutôt que livrer une donnée fausse.

4. Validation du programme

4.1 Script de test

Le fichier `hamming.py` a été testé avec le script fourni, sans aucune modification de celui-ci.

```
Q1  int2bits          OK
Q2  bits2int          OK
Q3  generate_message  OK
Q4  formatage         OK
Q5  XOR               OK
Q6  test_hamming      OK
Q7  set_correctors    OK
Q8  codage_hamming    OK
Q10 swap              OK
Q10 decodage          OK
Q11 parite            OK
Q12 codage_hamming2   OK
Q13 decodage2         OK

Fonctions validées : 13 / 13
Bonus (indicatif) : +0.50 point
```

4.2 Conformité au cahier des charges

Exigence du sujet	Réalisation
Fichier nommé `hamming.py`	oui, à la racine de `TP6`
Fonctions demandées en rose dans l'énoncé	les 13 fonctions sont présentes, aux noms exacts
Pas de variables globales	seules des définitions de fonctions, plus l'import `from random import randint`
Pas de tests dans le fichier	oui : les vérifications sont dans `test-hamming.py` et `verification.py`
Arguments d'entrée non modifiés	`formatage`, `set_correctors`, `swap` et `decodage` travaillent sur des copies
Types d'entrée/sortie	listes d'entiers 0 / 1, comme dans l'énoncé

Deux fonctions auxiliaires ont été ajoutées : `est_controle` et `positions_message`. Elles ne sont pas réclamées par le sujet, mais factorisent le test « position de contrôle » utilisé par quatre fonctions du TP, ce qui évite de répéter l'expression `i == 0 or (i & (i - 1)) == 0` et rend le code plus sûr. Elles restent de simples fonctions, sans état.

4.3 Complexités

Fonction	Complexité	Remarque
`int2bits`, `bits2int`	O(n)	n = nombre de bits
`generate_message`	$O(2^n)$	un tirage par bit
`formatage`	$O(2^n)$	une écriture par position
`XOR`	$O(2^n)$	terme à terme
`test_hamming`	$O(n \cdot 2^n)$	un XOR de n bits par 1
`set_correctors`	$O(n \cdot 2^n)$	dominé par le syndrome
`codage_hamming`, `codage_hamming2`	$O(n \cdot 2^n)$	deux passages
`swap`, `parite`	$O(2^n)$	copie / somme
`decodage`, `decodage2`	$O(n \cdot 2^n)$	un syndrome, plus une correction

Le coût est linéaire en la taille du mot de code, à un facteur n près pour le calcul du syndrome : le codage et le décodage sont donc **linéaires**, ce qui est la propriété recherchée pour un code correcteur utilisé en transmission à haut débit.

5. Bilan

Le TP couvre la chaîne complète d'un code correcteur : écriture binaire des indices (Q1 et Q2), génération et mise en forme (Q3 et Q4), arithmétique du XOR (Q5), calcul du syndrome (Q6), détermination des bits de contrôle (Q7 et Q8), puis décodage avec localisation (Q10), et enfin extension par un bit de parité pour détecter les erreurs doubles (Q11 à Q13).

Trois idées structurent l'ensemble. D'abord, **la position d'un bit s'écrit en binaire** : c'est ce qui transforme un XOR de positions en n équations indépendantes, une par bit de contrôle. Ensuite, **le syndrome est linéaire**, donc il s'additionne : syndrome d'un mot reçu = syndrome du mot émis ⊕ syndrome de l'erreur, ce qui explique que le syndrome « vaut » la position de l'erreur quand elle est unique. Enfin, **la parité globale lève l'ambiguïté d'arité** : elle ne dit pas où est l'erreur, mais elle dit combien il y en a, ce qui suffit à distinguer une erreur corrigeable d'un message à jeter.

La limite du procédé est la même que celle de tous les codes à distance finie : au-delà de la capacité de correction, le décodeur peut produire un message faux en croyant l'avoir corrigé. Le bit de parité de la question 12 transforme ce risque silencieux en un refus explicite (`None`), et c'est cette logique — corriger si l'on peut, refuser plutôt que mentir — que l'on retrouve dans les protocoles réels.

Annexe A — code complet de hamming.py

```
# -*- coding: utf-8 -*-
"""R3.09 - Cryptographie : code correcteur d'erreur de Hamming.

Codage d'un message de 2^n - (n + 1) bits en un mot de code de 2^n bits :
les positions 0, 1, 2, 4, ..., 2^(n-1) sont reservees aux bits de controle,
les autres accueillent les bits du message, dans l'ordre.

Le fichier ne contient que des definitions de fonctions (ni variable globale,
ni test) ; les noms et l'ordre des arguments sont ceux du sujet.
"""

from random import randint


def est_controle(i):
    """Vrai si la position i est reservee a un bit de controle.

    Ce sont la position 0 et les puissances de 2 (1, 2, 4, 8, ...).
    Un entier i > 0 est une puissance de 2 si et seulement si i & (i - 1)
    vaut 0 (le bit de poids fort de i est alors le seul allume).

    Entrees : i entier
    Sortie  : True si i est la position d'un bit de controle, False sinon
    """
    return i == 0 or (i & (i - 1)) == 0


def positions_message(n):
    """Liste des positions de message pour un mot de code de 2^n bits.

    Entrees : n entier (nombre de bits de controle)
    Sortie  : liste croissante des i de {0, ..., 2^n - 1} qui ne sont pas
              des positions de controle
    """
    return [i for i in range(2 ** n) if not est_controle(i)]


def int2bits(a, n):
```

```
    """Representation de a sur n bits, poids faible en premier.

    Entrees : a entier (0 <= a < 2^n), n nombre de bits
    Sortie  : liste de n bits, le k-ieme element etant le bit de poids 2^k

    Exemple : int2bits(13, 6) vaut [1, 0, 1, 1, 0, 0].
    """
    return [(a >> k) & 1 for k in range(n)]


def bits2int(bits):
    """Entier represente par un tableau de bits (poids faible en premier).

    Entrees : bits liste de 0 et de 1
    Sortie  : somme des bits[k] * 2^k ; c'est la reciproque de int2bits
    """
    return sum(b << k for k, b in enumerate(bits))


def generate_message(n):
    """Message aleatoire de taille 2^n - (n + 1).

    Entrees : n entier
    Sortie  : liste de 2^n - (n + 1) bits tires au hasard
    """
    return [randint(0, 1) for _ in range(2 ** n - (n + 1))]


def formatage(m, n):
    """Insertion des bits de controle (mis a 0) dans le message.

    Entrees : m liste de 2^n - (n + 1) bits, n entier
    Sortie  : nouvelle liste de 2^n bits, ou les bits du message occupent
              les positions qui ne sont pas des puissances de 2 et ou toutes
              les autres positions valent 0

    La liste m n'est pas modifiee.
    """
    formate = [0] * (2 ** n)
    for position, bit in zip(positions_message(n), m):
        formate[position] = bit
    return formate


def XOR(m, p):
    """Ou exclusif terme a terme de deux tableaux de meme taille.

    Entrees : m et p listes de bits de meme longueur
    Sortie  : liste [m[k] ^ p[k]] de meme longueur
    """
    return [a ^ b for a, b in zip(m, p)]


def test_hamming(m, n):
    """Syndrome de Hamming du mot m, ecrit sur n bits.

    On combine par XOR les representations int2bits(i, n) de toutes les
```

```
    positions i ou m vaut 1. Ce syndrome vaut 0 si le mot est un mot de
    code valide ; sinon il vaut (en binaire, poids faible en premier) la
    position du bit errone.

    Entrees : m liste de 2^n bits, n entier
    Sortie  : liste de n bits
    """
    syndrome = [0] * n
    for position, bit in enumerate(m):
        if bit == 1:
            syndrome = XOR(syndrome, int2bits(position, n))
    return syndrome


def set_correctors(m, n):
    """Calcul des bits de controle du mot formate m.

    Le k-ieme element de test_hamming(m, n) est la valeur a donner au bit
    de controle situe en position 2^k.

    Entrees : m liste de 2^n bits dont les bits de controle valent 0, n entier
    Sortie  : nouvelle liste de 2^n bits, bits de controle mis a jour
    """
    corrige = list(m)
    syndrome = test_hamming(corrige, n)
    for k in range(n):
        corrige[2 ** k] = syndrome[k]
    return corrige


def codage_hamming(m, n):
    """Codage de Hamming d'un message de 2^n - (n + 1) bits.

    Entrees : m liste de 2^n - (n + 1) bits, n entier
    Sortie  : mot de code de 2^n bits (message formate, bits de controle
              calcules), dont le syndrome est nul
    """
    return set_correctors(formatage(m, n), n)


def swap(m, k):
    """Copie de m dont le bit d'indice k a ete inverse.

    Entrees : m liste de bits, k indice valide de m
    Sortie  : nouvelle liste ; la liste m n'est pas modifiee
    """
    copie = list(m)
    copie[k] = 1 - copie[k]
    return copie


def decodage(m, n):
    """Decodage d'un mot de code de 2^n bits, avec correction d'une erreur.

    Le syndrome donne la position du bit errone (0 s'il n'y a pas d'erreur) :
    on inverse ce bit avant de relire le message.
```

```
    Entrees : m liste de 2^n bits, n entier
    Sortie  : liste de 2^n - (n + 1) bits, le message corrige
    """
    corrige = list(m)
    erreur = bits2int(test_hamming(corrige, n))
    if erreur != 0:
        corrige = swap(corrige, erreur)
    return [corrige[position] for position in positions_message(n)]


def parite(m):
    """Parite d'un tableau de bits.

    Entrees : m liste de bits
    Sortie  : 0 si le nombre de 1 est pair, 1 sinon
    """
    return sum(m) % 2


def codage_hamming2(m, n):
    """Codage de Hamming avec un bit de parite globale en position 0.

    Entrees : m liste de 2^n - (n + 1) bits, n entier
    Sortie  : mot de code de 2^n bits dont le syndrome est nul et dont le
              nombre de 1 est pair
    """
    code = codage_hamming(m, n)
    code[0] = parite(code)
    return code


def decodage2(m, n):
    """Decodage d'un mot code par codage_hamming2, avec detection d'erreur.

    Entrees : m liste de 2^n bits, n entier
    Sortie  : le message de 2^n - (n + 1) bits si l'on peut le reconstituer,
              None si au moins deux erreurs sont detectees

    Trois cas, selon le syndrome et la parite :
      - syndrome nul : seuls les bits du message et de controle sont surs,
        une parite fausse signifie que seul le bit 0 a ete modifie ;
      - syndrome non nul et parite fausse : une seule erreur, on la corrige ;
      - syndrome non nul et parite juste : au moins deux erreurs, rien a faire.
    """
    mot = list(m)
    erreur = bits2int(test_hamming(mot, n))
    if erreur == 0:
        return [mot[position] for position in positions_message(n)]
    if parite(mot) != 0:
        return decodage(mot, n)
    return None
```

Annexe B — traces d'exécution

```
================================================================================
Q1 - int2bits
================================================================================
  int2bits(13, 6  ) = [1, 0, 1, 1, 0, 0]     (13 = 1.2^0 + 1.2^2 + 1.2^3)
  int2bits(0, 4   ) = [0, 0, 0, 0]           (0 = 0)
  int2bits(15, 4  ) = [1, 1, 1, 1]           (15 = 1.2^0 + 1.2^1 + 1.2^2 + 1.2^3)
  int2bits(6, 3   ) = [0, 1, 1]              (6 = 1.2^1 + 1.2^2)
  int2bits(45, 6  ) = [1, 0, 1, 1, 0, 1]     (45 = 1.2^0 + 1.2^2 + 1.2^3 + 1.2^5)

================================================================================
Q2 - bits2int et reciprocite
================================================================================
  bits2int([1, 0, 1, 1, 0, 0]) = 13
  bits2int(int2bits(a, 8)) == a pour a = 0..255 : True
  int2bits(bits2int(x)) == x pour les 256 mots de 8 bits : True

================================================================================
Q3 - generate_message
================================================================================
  n = 3 -> 4 bits : [1, 1, 1, 0]
  n = 4 -> 11 bits : [0, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0]
  n = 5 -> 26 bits : [1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1,
         0]
  200 tirages pour n = 5 : 200 messages distincts

================================================================================
Q4 - formatage
================================================================================
  message      : [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]
  formatage    : [0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1]
  attendu      : [0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1]
  identique    : True
  positions de controle (n = 4) : [0, 1, 2, 4, 8]

================================================================================
Q6 - test_hamming
================================================================================
  positions des 1 de m    : [3, 7, 10, 11, 12, 14, 15]
  XOR de ces positions    : 3 ^ 7 ^ 10 ^ 11 ^ 12 ^ 14 ^ 15 = 8
  int2bits(8, 4) est bien [0, 0, 0, 1]
  test_hamming(m, 4)      : [0, 0, 0, 1]  (soit 8)
  test_hamming(code, 4)   : [0, 0, 0, 0]

================================================================================
Q7/Q8 - set_correctors et codage_hamming
================================================================================
  formatage(m, 4)             : [0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1]
  test_hamming(m, 4)          : [0, 0, 0, 1]
  set_correctors(m, 4)        : [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]
  codage_hamming(m, 4)        : [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]
  attendu                     : [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]
  identique                   : True
```

```
================================================================================
Q9 - verification : le syndrome d'un mot de code est nul
================================================================================
   n  taille m  taille code  tests  syndromes nuls
   3         4            8   1000            1000
   4        11           16   1000            1000
   5        26           32   1000            1000
   6        57           64   1000            1000
   7       120          128   1000            1000

  Nombre total de mots de code distincts verifies : n=3: 16  n=4: 2048  (total 2064)

================================================================================
Q10 - swap, syndrome et decodage
================================================================================
  code correct : [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]

  bit inverse  syndrome (entier)  message decode  correct
            0                  0  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
            1                  1  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
            2                  2  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
            3                  3  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
            4                  4  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
            5                  5  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
            6                  6  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
            7                  7  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
            8                  8  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
            9                  9  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
           10                 10  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
           11                 11  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
           12                 12  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
           13                 13  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
           14                 14  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui
           15                 15  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]      oui

  16 / 16 erreurs simples corrigees
  swap ne modifie pas son argument   : True

================================================================================
Q10 - ce que fait decodage face a deux erreurs
================================================================================
  bits inverses  syndrome                    decodage               resultat
       1 et 2            3  [0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]           MESSAGE FAUX
       0 et 1            1  [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]         message correct
       3 et 5            6  [0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1]           MESSAGE FAUX
       8 et 9            1  [1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1]           MESSAGE FAUX
      14 et 15           1  [1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0]           MESSAGE FAUX

================================================================================
Q11/Q12 - parite et codage_hamming2
================================================================================
  parite([0, 0, 0]) = 0
  parite([1, 0, 1, 1]) = 1
  parite([1, 1, 0, 1, 1]) = 0

  codage_hamming(m, 4)  : [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]   (8 bits a 1)
```

```
  codage_hamming2(m, 4) : [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]   (8 bits a 1)
  syndrome              : [0, 0, 0, 0]
  nombre de 1 pair      : True

  Second exemple, ou le bit de parite est necessaire :
    message               : [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
    codage_hamming        : [0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]   (3 bits a 1,
            impair)
    codage_hamming2       : [1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]   (4 bits a 1,
            pair)
    syndrome              : [0, 0, 0, 0]
    decodage2(c2, 4)      : [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

================================================================================
Q13 - decodage2 : 0, 1 et 2 erreurs
================================================================================
  mot de code : [0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1]
  0 erreur        -> [1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1]
  1 erreur (16 positions)  -> message retrouve : True
  2 erreurs (120 paires)   -> None : 120, autre : 0

================================================================================
Resume de l'execution du script de test
================================================================================
Q1  int2bits          OK
Q2  bits2int          OK
Q3  generate_message  OK
Q4  formatage         OK
Q5  XOR               OK
Q6  test_hamming      OK
Q7  set_correctors    OK
Q8  codage_hamming    OK
Q10 swap              OK
Q10 decodage          OK
Q11 parite            OK
Q12 codage_hamming2   OK
Q13 decodage2         OK

Fonctions validées : 13 / 13
Bonus (indicatif) : +0.50 point
```

Annexe C — reproductibilité

Le document et les résultats qu'il cite se régénèrent intégralement à partir du dépôt, sans outil externe (pas de LaTeX, pas de navigateur) :

Commande	Effet
`python3 test-hamming.py`	exécute les 13 tests fournis et affiche `13 / 13`
`python3 verification.py`	régénère les traces (question 9, erreurs simples et doubles, parité) dans `resultats-verification.txt`
`python3 md2pdf.py responses.md rendu.pdf`	reconstruit ce compte-rendu à partir de `responses.md`

Le fichier `hamming.py` ne contient que les fonctions demandées : il peut être déposé tel quel sur le Moodle. `verification.py` et `md2pdf.py` sont des outils de rédaction, ils ne font pas partie du rendu.

Les valeurs citées dans les tableaux de ce document proviennent toutes de `resultats-verification.txt`, lui-même produit par `verification.py` : aucune n'est recopiée à la main.